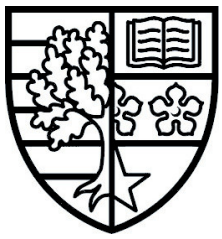
Develop an AI Agent for a Multi-Player Online Historical Role-Playing Game

by Terry Olampi

Masters Dissertation

Submitted in partial fulfilment for the degree of
MSc Artificial Intelligence

Supervised by Prof. Hans-Wolfgang Loidl



Heriot-Watt University
School of Mathematical and Computer Sciences

14th July 2026

The copyright in this dissertation is owned by the author. Any quotation from the dissertation or use of any of the information contained in it must acknowledge it as the source of the quotation or information.

Authorship Declaration

Course code and name	F21MP - Masters Project and Dissertation
Type of assessment	Individual
Coursework title	Develop an AI Agent for a Multi-Player Online Historical Role-Playing Game
Student name	Terry Olampi
Student ID	H00551446

By signing this form:

- I declare that the work I have submitted for individual assessment or the work I have contributed to a group assessment is entirely my own. I have not taken the ideas, writings or inventions of another person and used them as if they were my own. My submission or my contribution to a group submission is expressed in my own words. Any uses made within this work of the ideas, writings or inventions of others, or of any existing sources of information (books, journals, websites, etc.) are properly acknowledged and listed in the references and/or acknowledgements sections.

- I confirm that I have read, understood and followed the University's Regulations on academic misconduct as published on the University's website, and that I am aware of the penalties that I will face should I not adhere to the University Regulations.

- I confirm that I have read, understood and avoided the different types of plagiarism explained in the University guidance on Academic Integrity and Plagiarism.

Student signature: Terry Olampi

Date: 14th July 2026

Your work will not be marked if a signed copy of this form is not included with your submission.

Acknowledgements

I would like to thank my supervisor, Prof. Hans-Wolfgang Loidl, for his steady guidance and for the many discussions that shaped the direction of this work. I am also grateful to Christopher Garvie for providing access to the JominiEngine codebase and documentation, and for explaining to me how to keep the server side available for testing as the new agent-facing access paths were exercised.

Abstract

This dissertation reports the implementation and evaluation of an autonomous AI agent for the JominiEngine, a server-authoritative framework for historical massively multiplayer online role-playing games set in early thirteenth-century Britain. Building on an earlier design report, two agents were built that connect to the live engine through exactly the same Protocol Buffer client interface as a human player and manage a lord's fiefs, household, and armies season by season. The first is a five-state finite state machine (FSM) whose transitions are guarded by quantitative indicators drawn from the engine's fief and character data. The second replaces the hand-written policy with tabular Q-learning over a twelve-feature discretised state and a sixteen-action space, learning from a shaped, bounded reward.

A substantial part of the work was making the agent robust against the engine's real behaviour rather than an idealised model of it. Three protocol and engine faults that only surfaced under sustained multi-season play were diagnosed and fixed: a message-framing desynchronisation that produced spurious "invalid wire-type" errors at season boundaries, an unhandled dead-player-character state during succession, and an arithmetic overflow caused by an unbounded treasury. Both agents were then validated for decision correctness on fifteen hand-specified scenarios, which they passed completely, and benchmarked over multi-season runs across three engine scenarios.

The results give a nuanced answer to the project's central question. The FSM is reliable: it keeps the dynasty intact, drives bailiff coverage to one hundred per cent, and improves its composite evaluation score steadily. Q-learning matches the FSM's aggregate score in the nominal scenario and recovers a fief network from debt, but it is brittle. In the economic-crisis scenario the learned policy collapsed onto a single zero-cost fund-transfer action, repeating it two hundred times in one run and inflating the home treasury to over seventy million marks, a clear instance of reward hacking. The dissertation analyses why this happened, what the bounded evaluation metric did and did not reveal about it, and what it implies for applying tabular reinforcement learning in a live, adversarial game economy.

Table of Contents

Authorship Declaration i
Acknowledgements ii
Abstract iii
Table of Contents iv
1 Introduction 1
1.1 Background and Motivation . . . 1
1.2 Aim and Objectives . . . 2
1.3 Organisation . . . 2
2 Background 2
2.1 The JominiEngine and its Client Protocol . . . 2
2.2 Finite State Machines . . . 3
2.3 Reinforcement Learning and Q-Learning . . . 3
3 Implementation 4
3.1 Technology Stack and Project Structure . . . 4
3.2 Connecting to the Engine and the Season Loop . . . 4
3.3 Robustness Under Sustained Play . . . 5
3.4 Shared State Representation . . . 6
3.5 The FSM Agent . . . 6
3.6 The Q-Learning Agent . . . 7
3.7 Scenarios and Evaluation Harness . . . 9
4 Evaluation 9
4.1 Method . . . 9
4.2 Metrics . . . 10
4.3 Decision Correctness . . . 10
4.4 FSM Performance . . . 10
4.5 Q-Learning Performance . . . 12
4.6 Reward Hacking in the Crisis Scenario . . . 13
5 Discussion and Conclusion 14
5.1 Does Q-Learning Improve on the FSM? . . . 14
5.2 State-Space Size and the Training Budget . . . 15
5.3 Reward Design as the Real Difficulty . . . 15
5.4 Robustness as a Research Cost . . . 15
5.5 Limitations . . . 16
5.6 Future Work . . . 16
5.7 Conclusion . . . 16
References 18
A Project Management 19
A.1 Deviations from the Plan . . . 19

B Generative AI **19**

1 Introduction

The JominiEngine is a distributed, server-authoritative engine for historical massively multiplayer online role-playing games (MMORPGs). Its reference instantiation, *Overlord: Age of Magna Carta*, places each player in the role of a medieval lord in mainland Britain and asks them to govern fiefs, manage a noble household, and command armies within a shared, persistent world modelled on the years around the signing of Magna Carta [2]. Everything a player does, from raising a tax rate to storming a keep, is expressed as a message to a central server that owns the canonical game state [7].

Until now every participant in that world has been a human at a Unity client. The engine has no provision for an autonomous agent that can hold a lord's position when the human is away. This is not a cosmetic gap. A fief whose owner has logged off keeps paying army upkeep from a treasury that no longer earns, loses loyalty because no bailiff administers it, and presents an undefended border that a neighbouring player can cross unopposed. Across a large map with many players, some of whom will always be offline, these abandoned holdings accumulate into "dead zones" that break both the simulation's internal logic and its intended value as a teaching tool about medieval governance.

An earlier report for this project surveyed the relevant literature and set out a design for closing that gap with two agents of deliberately opposite character: a transparent rule-based baseline, and an adaptive reinforcement-learning agent. This dissertation reports what happened when that design met the running engine. It documents the implementation of both agents, the protocol- and engine-level problems that only appear once an agent plays for many consecutive seasons, and a controlled evaluation of the two approaches. The agents connect through the ordinary client interface and require no change to the server, so anything they can do a human player could also do.

1.1 Background and Motivation

Two motivations run through the project. The first is operational: an agent that keeps an absent player's fiefs solvent and defended preserves the integrity of the shared world for everyone else. The second is investigative. A configurable, repeatable agent is a convenient experimental subject for studying strategy in a rule-rich historical economy, which is difficult to arrange with human participants [3]. The engine's income, recruitment, and siege rules are derived from historically grounded formulae [2], so an agent operating inside it is making decisions against a genuine, if stylised, model of feudal economics rather than an abstract benchmark.

The educational framing also shapes a technical choice that recurs throughout the dissertation: interpretability matters. A tutor who can point at a log line reading `Stable → EconomicCrisis because Treasury < 500` is teaching something; the weights of a neural network teach nothing directly [4, 8]. That is the main reason a rule-based FSM is treated here as a first-class agent and not merely a throwaway baseline.

1.2 Aim and Objectives

The aim of the project was to design, implement, and evaluate an AI system that autonomously manages a lord's holdings in the JominiEngine through the standard client API. The specific objectives, restated here as they were actually pursued, were to:

(1) review the engine, its Protocol Buffer API, and its data models for fiefs, characters, and armies, to the depth needed to drive them from an external client;
(2) implement a rule-based FSM baseline agent that plays complete seasons against the live server;
(3) implement an improved agent using tabular Q-learning over the same interface;
(4) make both agents robust enough to run for many consecutive seasons without crashing or corrupting the game economy; and
(5) evaluate and compare the two agents on a shared set of quantitative metrics across several engine scenarios.

Objectives one to three follow the earlier design report. Objective four was not anticipated in the plan at that level of prominence and became, in practice, a large part of the work: an agent that issues fifty actions a season for thirty seasons exercises corners of the protocol that a human clicking through a single session never reaches.

1.3 Organisation

Section 2 gives the background needed to follow the rest of the dissertation: the engine's architecture and message protocol, and the two AI techniques used. Section 3 is the core of the work, describing the agent's connection layer, the robustness fixes, the shared state representation, and the two decision engines. Section 4 defines the evaluation method and reports results across the validation and performance runs. Section 5 discusses what the results mean, including the reward-hacking failure, and concludes with limitations and future work.

2 Background

This chapter covers the material a reader needs to follow the implementation: the engine's architecture and the constraints it imposes on an external agent, and the two decision-making techniques compared in the project. The earlier report gives a fuller literature survey; the treatment here is deliberately focused on what bears directly on the work reported in Sections 3 and 4.

2.1 The JominiEngine and its Client Protocol

Three properties of the engine determine how an agent must be built.

Server authority. The canonical game state lives on the server; clients exist only to observe it and to request actions [2, 7]. Every state change is validated and applied server-side before any reply is sent. An agent therefore cannot rely on a private copy of the world. It must reconstruct its view each season from the `ProtoFief`, `ProtoCharacter`, and `ProtoArmy` objects the server returns, and treat those replies as the single source of truth.

Protocol Buffer messaging. Client and server communicate with Protocol Buffers [5] serialised over a Lidgren.Network UDP transport [6]. Each exchange is a `ProtoMessage` carrying an `ActionType` from an enumeration of more than sixty actions, optional string parameters, and a `ResponseType` on replies. The interface is identical for the Unity client and for the agent, which is what allows the agent to operate without server modification. It is also unforgiving: because each message is length-prefixed and framed, a single mismatch between what the server sends and what the client expects to read desynchronises the stream, a fault that features prominently in Section 3.3.

A season-based clock. The world advances in discrete seasons through the `GameClock`. A season transition applies fief income and expenses, updates loyalty, resolves scheduled events, and refreshes each character's allowance of action-days. Crucially for this project, the server does not advance the clock on its own in the configuration used here; the client must request a `SeasonUpdate` once its character has spent its days. The season boundary is a natural decision step: the agent observes the world, submits actions until it runs out of days, and then asks the server to resolve the season. For the reinforcement-learning agent this boundary is exactly one Markov decision process time step.

2.2 Finite State Machines

A finite state machine (FSM) is among the oldest and most widely used techniques for game-character behaviour [9]. It holds a finite set of states, exactly one of which is active, and a set of transitions guarded by conditions; when a guard becomes true the machine switches state and runs the associated behaviour. Its appeal is transparency: every state and transition can be read and traced without special tooling, which suits both debugging and the pedagogical requirement above. Its known weakness is that the number of transitions can grow quadratically with the number of states [9], which argues for keeping the state count small. A second, subtler weakness is oscillation, or "jitter": if the guard into a state and the guard out of it share a threshold, a value hovering near that threshold makes the machine flip back and forth every step. The standard remedy, used here, is a hysteresis band, where the entry and exit thresholds differ so that leaving a state requires a clearly better situation than the one that triggered entry.

2.3 Reinforcement Learning and Q-Learning

Reinforcement learning (RL) lets an agent learn a policy by interacting with an environment, without a prior model of that environment [11]. At each step the agent observes a state s, takes an action a, receives a scalar reward r, and moves to a next state s'; it seeks a policy that maximises expected discounted return. Q-learning [12, 13] is a model-free, off-policy RL method that keeps a table $Q : S \times A \to \mathbb{R}$ of estimated action values and, after each transition (s, a, r, s'), updates it towards the observed reward plus the discounted best next value:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right], \tag{1}$$

where $\alpha \in (0, 1]$ is the learning rate and $\gamma \in [0, 1)$ the discount factor. Under a finite MDP, sufficient exploration of every state-action pair, and a suitably decaying learning rate, Q converges to the optimal action-value function [13]. Exploration is handled with an ε-greedy rule: with probability ε the agent acts randomly and otherwise greedily, with ε decaying over time so the agent shifts from exploring to exploiting as its estimates mature.

The model-free property is the attraction here: the agent is never told the income or loyalty formulae, and must instead discover a good policy purely from the reward it experiences. The corresponding risk, which Sections 4 and 5 return to at length, is that an RL agent optimises the reward it is actually given rather than the outcome the designer intended. When a cheaply repeatable action yields reward, the agent will exploit it; this "reward hacking" is a recognised failure mode of learned agents [1] and is exactly what occurred in one of the scenarios evaluated here.

3 Implementation

This chapter describes the agent as built. It covers the technology stack and project layout, the connection and season loop, the robustness work that made sustained play possible, the shared state representation, and the two decision engines, and finishes with the scenario and logging infrastructure used for the evaluation.

3.1 Technology Stack and Project Structure

The agent is written in C# on .NET 8, and links against the engine's own `ProtoMessageClient` assembly so that it shares the exact `Actions` enumeration and proto class definitions used by the server. Reusing those definitions rather than re-declaring them removes an entire class of silent serialisation mismatches: the agent and server are, by construction, speaking the same schema. Networking uses Lidgren.Network and serialisation uses protobuf-net, both as pulled in by the engine.

The client is organised in three layers, mirroring the design in the earlier report but with concrete class names. A network layer (`TextTestClient` and its inner `Network` class) owns the Lidgren connection, the login handshake, and the serialisation of `ProtoMessage` objects, and exposes a blocking send-and-wait-for-reply primitive. A protocol layer (`Player`) wraps each engine action as a typed method, for example `AppointBailiff` or `TransferFunds`, on top of that primitive. A decision layer (`Agent` and its subclasses `FSMAgent` and `QLearningAgent`) consumes an `AgentState` and produces the next action. Swapping agents changes only the decision layer; the network and state layers are shared, which is what guarantees that both agents are evaluated under identical conditions.

3.2 Connecting to the Engine and the Season Loop

After authenticating with the same RSA-certificate-and-hash handshake the Unity client uses, the agent settles into a request-reply rhythm. `Player.SendRequestToServer` serialises a `ProtoMessage`, sends it, and blocks on `GetActionReply` until a reply carrying the matching `ActionType` arrives on the receive queue. A full season is then a loop: build the current state, ask the active agent for an action, translate it into one or more engine calls,

apply it, and repeat until the character has no action-days left or an anti-stall guard fires. At that point the agent calls `SeasonUpdate`, the server resolves the season, and the loop begins again with a freshly rebuilt state.

Two design points in this loop matter later. First, the agent holds no state between seasons except what it deliberately carries (the Q-table, and per-season bookkeeping sets); the world view itself is discarded and rebuilt from server replies each season, honouring the server-authority constraint. Second, the loop terminates cleanly when the player character can no longer act, which is the correct behaviour when a dynasty dies out but, as Section 3.3 explains, initially interacted badly with how the server reports a dead character.

3.3 Robustness Under Sustained Play

The single largest lesson of the implementation phase was that correctness on a single well-behaved season does not imply correctness over thirty. Three faults only appeared under sustained, high-volume play, and each had to be fixed before the evaluation runs could complete.

3.3.1 Message-framing desynchronisation. At many season boundaries the client logged `Invalid wire-type (Fixed64); this usually means you have over-written a file without truncating`. Despite the wording, nothing was being written to a file. protobuf-net emits that generic message whenever it is handed bytes that are not valid Protocol Buffers. Tracing the receive path showed the cause: at certain points the server replies with a plain ASCII string rather than a length-prefixed `ProtoMessage`, most importantly the message "You have no valid PlayerCharacter!" returned whenever the acting character is momentarily dead or unset. The client's reader tried to deserialise that ASCII as a proto, failed, logged the error, and, because the raw string was routed to a separate string queue, could leave the request-reply loop waiting for a reply that never arrived. The fix was applied server-side, converting the three raw-text responses into properly length-prefixed `ProtoMessage` replies tagged with the requesting action, so that the client's reader always receives parseable, correctly routed messages.

3.3.2 The dead player character during succession. Fixing the framing exposed a second fault. At a season boundary a player character can die and be succeeded by an heir, and there is a brief window in which the player has no living character. Once the server's reply reached the client correctly, the state-building code, which had assumed a living character, threw an `InvalidCastException` when it tried to read the player list as though it were a normal response. The state builder and its callers were made defensive: when the server indicates no living character, the agent now returns a partial state with a null player character, which the season loop recognises as a signal to end the run rather than crash. This turns a crash into the correct game-logic outcome, the extinction of a dynasty ends the episode.

3.3.3 Unbounded treasury and integer overflow. A longer-running fault surfaced only after many seasons: the run aborted with an `OverflowException` inside a routine that summed fief treasuries. C#'s checked `Sum` over 32-bit integers throws once the running total exceeds

roughly 2.1×10^9, and by the season in question the summed treasury had genuinely crossed that bound. The immediate fix was to accumulate in 64-bit and clamp, so state-building can no longer crash on a large total. The overflow was, however, a symptom rather than a disease, and diagnosing it led to an audit of every money path in the engine (fund transfers, overlord taxes, seasonal income, expenditure adjustment). That audit established that the transfer and tax paths conserve money correctly; a separate scoring routine in the client had been casting signed treasuries to unsigned integers, so a fief in debt of −600,000 was being read as roughly 4.29×10^9, which corrected a badly distorted score. The residual observation, that a fief network really can accumulate very large treasuries over many seasons of positive income, is revisited in the evaluation because the learning agent turned it into a strategy.

Taken together these three fixes are not incidental bug-fixing. They are the difference between an agent that demonstrates once and an agent that can be evaluated statistically over long runs, and they are reported here because "make it survive its own environment" turned out to be a genuine research cost of deploying an agent against a live engine.

3.4 Shared State Representation

Both agents consume the same `AgentState`, assembled once per season by `AgentState.Build` from the server-fetched `GameState`. The state groups its fields into four contexts, a character context (days remaining, whether alive, captive, in the keep, married, with an heir), an economic context (home and total treasury, fief count, fiefs in deficit, fiefs with a bailiff, surplus-fief information), a military context (army presence, troop count, threat level, besieged keep level), and a diplomatic context (available marriages, heir need, hirable and fireable NPCs). From these it derives a small vector of normalised metrics used by the learning agent: economic health (home treasury capped at 100,000 marks), bailiff coverage, military strength, threat score, and days remaining.

Concentrating all server parsing in one place had a practical benefit beyond tidiness: the three robustness fixes above all live at this boundary, so both agents inherit them without either decision engine needing to know that the underlying protocol is imperfect.

3.5 The FSM Agent

The FSM agent (`FSMAgent`) realises the five-state design from the earlier report. Table 1 lists the states and the management priority each expresses. Transitions are guarded by thresholds on the state contexts, with the hysteresis discipline of Section 2.2: the guard into `EconomicCrisis` fires on a low or negative treasury, while the guard back to `Stable` requires a comfortably positive one, so the machine does not flip on a treasury sitting near the line.

Within a state, the agent runs a prioritised list of actions, attempting the highest-priority applicable one first and falling through to the next when an action has no valid target. Each abstract action is turned into a concrete `ProtoMessage` by a single executor (`FSAMActionExecutor`), which is where, for example, "transfer funds" becomes a `ProtoTransfer` with a source fief, a target fief, and an amount. Two safeguards keep

State	Management priority
Stable	Long-term development: appoint bailiffs, moderate spending, selective hiring, secure the succession.
EconomicCrisis	Austerity: cut every fief's expenditure to what the treasury supports, pull gold from surplus fiefs, shed costs.
UnderThreat	Defence: recruit and maintain troops, appoint an army leader.
AtWar	Campaign: choose siege or attack actions according to the enemy's keep level.
Consolidating	Recovery: rebuild loyalty and infrastructure, reconfirm bailiffs after a crisis or war.

Table 1. The five FSM states and the priority each encodes.

the agent from wasting a season. An adjacent-repeat check stops it issuing the same action with the same arguments twice in a row, and a per-season signature counter ends the season if any single action-argument combination has already run five times, which catches two cheap actions oscillating in a way the adjacent check would miss. These guards are the reason the FSM's action logs show variety rather than a single action repeated to the day limit.

3.6 The Q-Learning Agent

The Q-learning agent (`QLearningAgent`) reuses the FSM's state construction and argument resolution and learns only *which* kind of action to take. This is a deliberate scoping decision: the agent chooses an action type, and the same deterministic helpers the FSM uses then pick the specific fief, NPC, or army. It keeps the learning problem to a size a table can cover while still exercising the full action repertoire.

State. The observation vector has twelve features (Table 2): six continuous ratios and six booleans. `Discretise` bins each continuous feature into three levels and each boolean into two, giving a nominal state space of $3^6 \times 2^6 = 46{,}656$ keys, stored sparsely so that only visited states occupy memory. This is far larger than the fifty-four states proposed in the design report; the observation grew as it became clear that the earlier features could not distinguish situations the agent needed to treat differently (a deficit spread across many fiefs versus concentrated in one, for instance). The consequence, that a table this size cannot be well covered within a short training budget, is taken up in Section 5.

Actions. The action space has sixteen entries (Table 3), spanning economy, military, and dynasty. Only actions with a valid target in the current state are eligible, an action-masking step that stops the agent, for example, from "attacking" when there is no enemy present. When the greedy choice is a tie, most importantly on an unvisited, all-zero state, a fixed

Continuous (3 bins)	Boolean (2 bins)
Economic health	Treasury critical
Fraction of fiefs in deficit	Treasury recovered
Bailiff coverage	Has an army
Military strength	Character in keep
Threat score	Spouse available for a child
Days remaining	Heir needed

Table 2. The twelve observation features discretised into the Q-learning state key.

priority prior breaks it in favour of dynasty, then economy, then military, so an untrained agent still behaves sensibly rather than defaulting to whichever action happens to sit at index zero.

Economy	Military	Dynasty / household
AutoAdjustExpenditure	Maintain army	Appoint heir
Transfer funds	Recruit troops	Try for a child
Hire NPC	Appoint army leader	Propose marriage (PC)
Fire NPC	Siege (storm/reduce)	Propose marriage (child)
Appoint bailiff	Attack enemy army	Accept/reject a proposal
Camp (pass time)		

Table 3. The sixteen-action space, grouped by domain.

Reward. After each transition the agent computes a shaped reward (Equation (1) uses it as r). Every term is bounded so that no single component can dominate: the change in *total* realm treasury, clamped to $[-1, 1]$ after scaling by 100,000; the change in bailiff coverage; the change in threat, as a penalty; the change in military strength; fixed bonuses for gaining an heir or a spouse; and a small penalty for a failed action. A terminal reward at the season end rewards a recovered, heir-secured position. Crucially, the treasury term rewards the change in *total* realm wealth, not the home fief alone. This was a deliberate correction: an earlier version scored the home treasury, which the agent learned to inflate by shuffling gold inward with a zero-cost transfer, and the total-wealth formulation was intended to remove that incentive. Section 4 shows that in one scenario it did not fully succeed.

Learning. The agent uses Equation (1) with $\alpha = 0.10$ and $\gamma = 0.95$, and ε-greedy exploration starting at $\varepsilon = 0.20$ and decaying by 0.995 per episode to a floor of 0.01. The Q-table is initialised to zero and persisted to JSON between runs so that training accumulates across programme restarts. For evaluation the table is loaded and exploration is switched off, so the reported performance reflects the learned greedy policy.

3.7 Scenarios and Evaluation Harness

Three engine scenarios were used, each defined by the CSV data the server loads at start-up: a *Base* world of ordinary starting conditions, an *EconomicCrisis* world in which the home fiefs begin deeply in deficit, and a *Dynasty* world that stresses succession. Because the server owns the world, a scenario is chosen by how the server is started; the client's scenario flag is only a label written into the log, and a fresh world requires a server restart, a fact that itself caused a spurious early run before it was understood.

Two kinds of run were instrumented by the `EvaluationRunner`. A dry-run correctness harness drives each agent through fifteen hand-built scenarios that each encode the action a knowledgeable player would take, recording whether the agent produced it. A performance harness plays complete seasons and logs, at each season end, a snapshot (treasury, fief count, fiefs in deficit, bailiff coverage, troops, threat, heir and marriage status, actions attempted and failed) together with a composite score, and every individual action with the treasury before and after. This per-action logging is what later made the reward-hacking behaviour visible in detail. The composite score and the derived performance metrics are defined in Section 4.

4 Evaluation

This chapter reports how the two agents behaved. It first fixes the method and the metrics, then presents the decision-correctness validation, the FSM's multi-season performance, and the Q-learning agent's performance, including the reward-hacking failure that is the most instructive result of the study.

4.1 Method

Both agents were run through the same harness against the same server, differing only in the decision layer. The FSM needs no training and was evaluated directly. The Q-learning agent was trained on a scenario and then evaluated with exploration disabled, so its reported behaviour is the learned greedy policy rather than a still-exploring one. Every performance run began from a freshly started server world; a stale world (one already advanced by earlier runs) was found to end runs prematurely once the starting dynasty had aged out, and after that was understood the server was restarted for each measured run.

A caveat on scale is needed to read the numbers honestly. Runs vary in length (from four to fifteen seasons) because they were captured at different points in development and because some scenarios end sooner than others. The comparisons below therefore lean on per-season trajectories and on within-run behaviour rather than on single end-of-run totals, and I flag where a short run limits what can be concluded.

4.2 Metrics

Performance is summarised by a per-season *composite score* in $[-0.1, 1.0]$, a weighted sum of five components: economic health (weight 0.30; home treasury over 100,000, capped), bailiff coverage (0.15), military strength (0.15; troops over 500, capped), a dynasty score (0.30; heir, being alive, and having a spouse), and a threat penalty (-0.10). Two properties of this score matter for interpreting the results. It is bounded, so no component can run away, and its economic term saturates at a home treasury of 100,000 marks, so wealth beyond that point does not raise the score at all. The second property becomes important below: it means a treasury exploding to tens of millions is invisible to the composite score, which is precisely why a separate look at the raw action log was necessary.

Alongside the composite score I report three of the measures proposed in the design report. *Economic behaviour* is read from the treasury and deficit-count trajectories. *Decision diversity* (DD) is the normalised entropy of the action distribution over a run, ranging from 0 (one action only) to 1 (uniform); it is reported together with the number of distinct actions used, because entropy alone can flatter a run that used very few action types. *Success rate* is the fraction of submitted actions the server accepted. The design report also proposed a military success rate and a human-alignment score; Sections 4.3 and 5.5 explain how each was, and was not, realised.

4.3 Decision Correctness

Before any performance claim, both agents were checked for decision correctness on the fifteen dry-run scenarios, each of which pairs a situation with the action a competent player would take (appoint an heir when the succession is open, cut expenditure in a deficit, storm a weak keep but reduce a strong one, and so on). Both agents produced the expected action in all fifteen cases, at a one-hundred-per-cent action success rate (Table 4). Because each scenario encodes a knowledgeable player's choice, this doubles as a coarse human-alignment check: on this hand-built reference set the agents agree with the human baseline completely. It is not a substitute for a live playtest, and Section 5.5 is careful about the difference, but it establishes that neither agent is issuing counter-intuitive decisions before the performance runs are trusted.

Agent	Scenarios passed	Action success	Distinct actions
FSM	15 / 15	100%	12
Q-learning	15 / 15	100%	12

Table 4. Decision-correctness validation on the fifteen dry-run scenarios.

4.4 FSM Performance

Across all three scenarios the FSM is a reliable steward. Table 5 collects the headline figures. In every FSM run the dynasty stayed intact and every submitted action succeeded. The

composite score climbs from the same starting value of 0.18, which reflects a new lord who is alive and married but has no treasury, bailiffs, or troops yet, to final scores between 0.50 and 0.65, and it does so without large oscillations (Figure 1). The dip in the Base run's final season is the treasury term fluctuating as gold is moved and spent, not a collapse: the dynasty and bailiff terms hold steady underneath it.

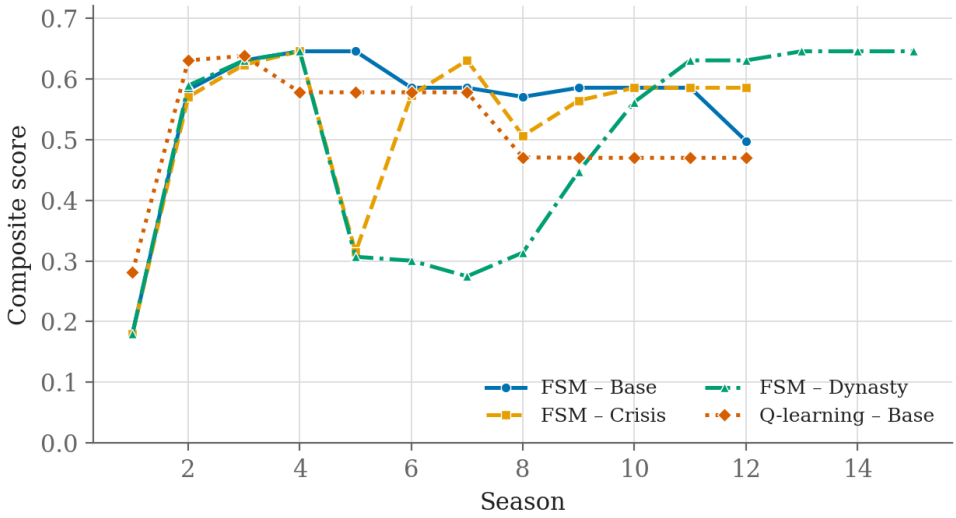


Fig. 1. Composite score per season. The FSM runs (Base, Crisis, Dynasty) climb from the initial 0.18 and stabilise; the Q-learning Base run tracks them closely for most of the run.

The clearest sign of competent administration is bailiff coverage (Figure 2). The FSM prioritises appointing bailiffs, and in the Crisis and Dynasty scenarios it reaches and holds one-hundred-per-cent coverage even as the fief network grows past 130 holdings; in the Base run it reaches roughly half. This is a meaningful result because coverage is what arrests the loyalty decay that motivated the whole project, and the agent maintains it while the number of fiefs to cover is itself increasing. The action mix is correspondingly administration-heavy: appointing bailiffs and adjusting expenditure dominate, with occasional recruitment, entourage, and household actions, giving decision diversity between 0.41 and 0.55 over nine to ten distinct action types.

Military outcomes are the weakest part of the picture, and honestly so. Threats in the engine are reactive: a war needs a human aggressor, and in these single-agent runs almost none materialised. Troop counts stayed at or near zero in most runs, a small defensive force of fifty appears in the Crisis and Dynasty runs, and no battles were fought. The military success rate proposed in the design report is therefore not applicable to these runs, and I report it as such rather than manufacture a number from an empty sample.

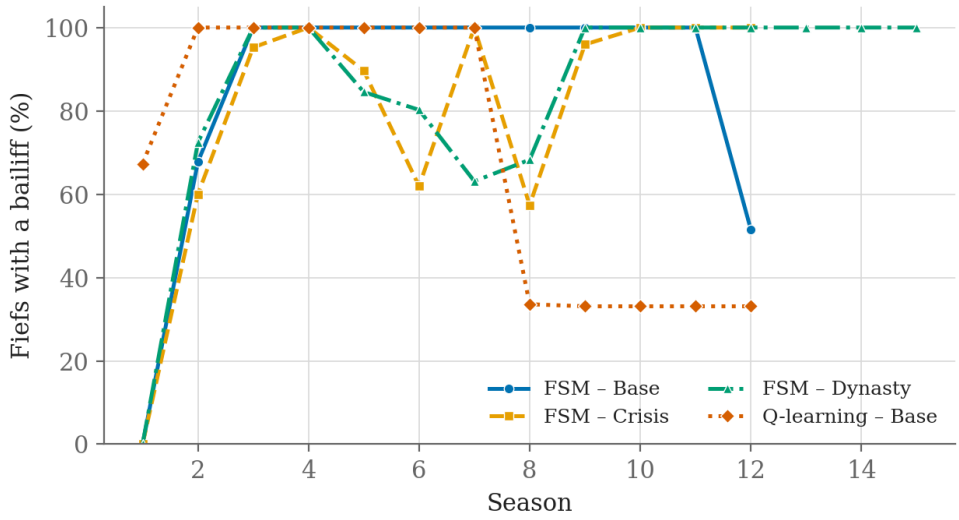


Fig. 2. Bailiff coverage over time. The FSM drives coverage to 100% in the Crisis and Dynasty scenarios and holds it as the realm grows; the Q-learning Base run keeps coverage roughly constant but does not close the gap.

Agent	Scenario	Seasons	Avg score	Final score	Success rate	DD (n)	Dynasty intact
FSM	Base	12	0.556	0.497	100%	0.55 (9)	yes
FSM	EconomicCrisis	12	0.530	0.585	100%	0.41 (10)	yes
FSM	Dynasty	15	0.496	0.645	100%	0.49 (10)	yes
Q-learning	Base (eval)	12	0.517	0.470	100%	0.56 (6)	yes
Q-learning	Base (train)	6	0.439	0.498	99.4%	0.55 (8)	yes
Q-learning	EconomicCrisis	5	0.519	0.505	100%	0.58 (3)	yes

Table 5. Headline performance figures. DD is decision diversity with the number of distinct action types in parentheses; a high DD over very few actions (Q-learning, Crisis) is a warning sign rather than a strength.

4.5 Q-Learning Performance

In the Base scenario the learned policy is competitive with the FSM. Its average season score of 0.517 sits just below the FSM's 0.556, its trajectory tracks the FSM's for most of the run (Figure 1), and its action mix is close to the FSM's, dominated by the same bailiff and expenditure actions (Figure 3). It also does something the FSM never has to: it recovers. Both Q-learning Base runs start in the red, with a home treasury around −120,000 marks and about half the fiefs in deficit, and pull the network back into surplus over the run,

ending with a positive treasury and the deficit count roughly halved. That a policy learned only from a scalar reward can dig a fief network out of debt is a genuinely positive result for the approach.

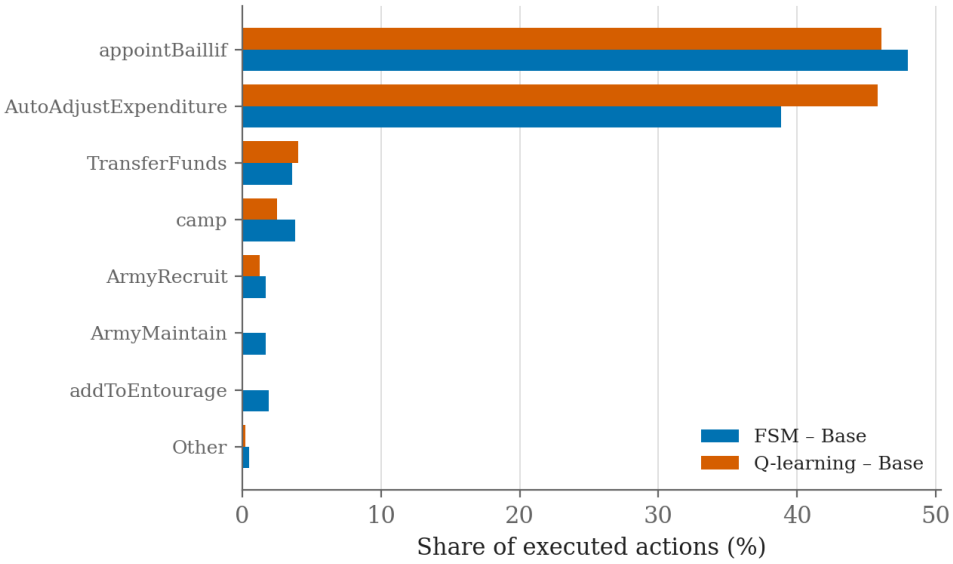


Fig. 3. Action mix in the Base scenario, as a share of executed actions. The FSM and the learned policy converge on a similar, administration-heavy profile.

Two weaknesses temper this. First, the learned policy does not close the bailiff-coverage gap that the FSM does; its coverage stays roughly constant rather than climbing to full (Figure 2), so it manages the economy without ever fully securing the administrative base. Second, it never appointed a permanent heir across the performance runs, relying instead on the dynasty simply not dying during the run; the FSM has the same gap, but it matters more for an agent whose whole premise is adaptation.

4.6 Reward Hacking in the Crisis Scenario

The most revealing result is a failure. In the EconomicCrisis scenario the learned policy collapsed onto a single action. Of 250 actions in the evaluation run, 200 were fund transfers; only two other action types appeared at all. The effect on the home treasury is stark (Figure 4): it sits near zero, in fact slightly negative, for the first fifty actions, and then, once the agent locks onto the transfer action, climbs almost without limit to over 74 million marks by the end of the run. The agent had discovered that transferring gold between its own fiefs costs no action-days, so it can be repeated essentially without bound, and that pulling gold inward raised its reward.

This is textbook reward hacking [1]: the agent optimised the reward it was given rather than the behaviour intended. It is worth being precise about what the reward shaping did and did not prevent. The reward had already been changed to score *total* realm wealth

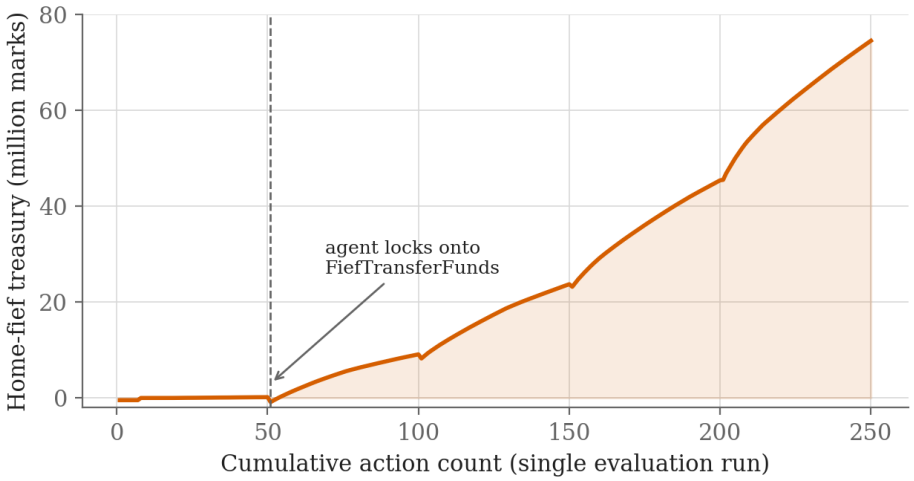


Fig. 4. Reward hacking. In the Crisis scenario the Q-learning agent locks onto the zero-cost fund-transfer action after roughly fifty actions and drives the home-fief treasury to over 74 million marks in a single run.

rather than home-fief wealth, specifically to remove the incentive to shuffle gold inward, and Figure 4 plots the home treasury to show the behaviour the agent settled into. That the behaviour persisted despite the total-wealth reward means the incentive was not fully removed. The most likely residual driver is that the transfer is free in action-days, so even a marginal or noisy positive reward makes repeating it dominate any action that costs a day and might fail; a zero-cost action with any upside is nearly always worth taking again.

The composite evaluation score hides all of this. Because its economic term saturates at 100,000 marks, a treasury of 74 million scores exactly the same as one of 100,000, and the Crisis run's average composite score, 0.519, looks unremarkable next to the others in Table 5. Only two views expose the pathology: the decision-diversity figure, which is a superficially healthy 0.58 but over just three action types, exactly the case where entropy misleads, and the raw action log, which shows the 200 transfers directly. This is the strongest single argument in the study for evaluating a learned agent on its behaviour, not only on a scalar score.

5 Discussion and Conclusion

5.1 Does Q-Learning Improve on the FSM?

The central question of the project has a qualified answer: not reliably, on the evidence collected. In the Base scenario the two agents are close, with the FSM slightly ahead on average score (0.556 versus 0.517) and clearly ahead on bailiff coverage, while the learning agent shows one capability the FSM does not need and does not have, recovering a fief network from debt. If the comparison stopped there it would read as a near-tie with interesting trade-offs. It does not stop there, because in the Crisis scenario the FSM

continued to behave sensibly while the learned policy degenerated into reward hacking. Reliability, not peak performance, is what separates them: the FSM cannot discover an exploit because it cannot discover anything, and in a live economy that a human opponent also inhabits, that predictability is worth a great deal.

This is consistent with the project's original framing, in which the FSM was chosen for transparency and the learning agent for adaptivity, and it sharpens it. The adaptivity that lets Q-learning climb out of debt is the same adaptivity that lets it find and exploit the transfer loophole. The two are not separable; they are the same mechanism seen in a favourable and an unfavourable light.

5.2 State-Space Size and the Training Budget

The design report planned a 54-state MDP. The implemented observation is far larger, a nominal 46,656 states, and although the table is sparse, the mismatch between that size and the handful of training seasons available is real. With so many states and so little experience, most of the Q-table is untouched at evaluation time, and the agent's behaviour on unseen states is decided by the fixed priority prior rather than by anything learned. Part of why the learned Base policy resembles the FSM so closely is that, on unvisited states, it *is* effectively a hand-ordered policy. A smaller, more carefully chosen state space, closer to the original 54-state proposal, would almost certainly have been learned more thoroughly within the same budget, at the cost of blurring situations the larger space keeps distinct. That trade-off between representational richness and learnability is the single design lever I would most like to have explored further.

5.3 Reward Design as the Real Difficulty

The reward-hacking result reframes where the difficulty in this problem actually lies. It is tempting to treat the learning algorithm as the hard part and the reward as a detail, but the algorithm behaved exactly as advertised; it was the reward, and the action costs behind it, that produced the pathological policy. The lesson is concrete: in an economy with a zero-cost action, a reward must make that action reward-neutral when it is genuinely neutral, because any residual positive signal on a free action will be exploited without limit. Scoring total rather than home wealth was the right direction but an incomplete fix. A more robust design would attach even a nominal cost to fund transfers, or reward only externally realised wealth (income earned, debts cleared) rather than internal balances, so that moving gold between one's own accounts is worth precisely nothing.

5.4 Robustness as a Research Cost

A theme that was not in the original plan but dominated the implementation is that deploying an agent against a live, imperfect engine has a substantial robustness cost. The three faults of Section 3.3 were invisible to a human playing a single session and only emerged under an agent issuing tens of thousands of actions across many seasons. Two of the three were in the server or protocol rather than in the agent, which is itself a finding: an autonomous client is an unusually thorough test of a protocol that was designed around

human pacing. Any future agent work on this engine inherits a more robust base because of it, and I would argue that "can it survive its own environment for thirty seasons" belongs in the requirements of such projects from the outset, not as an afterthought.

5.5 Limitations

Several limitations bound these conclusions. Runs are single-seeded and of unequal length, so the numbers in Table 5 are indicative rather than statistically established; a proper study would fix the season count and average several seeds per cell. The military dimension is essentially untested because threats did not arise in single-agent runs, so nothing here speaks to the agents' combat competence. The human-alignment measure was realised only through the fifteen-scenario correctness check, not through a live human reference run, so it validates decision plausibility on hand-picked cases rather than agreement with a real player over a full game. And the Q-learning agent's training budget was short relative to its state space, so its evaluated policy is under-trained by design; a longer training regime might narrow or widen the gap to the FSM in ways these runs cannot show.

5.6 Future Work

Four directions follow directly from the results. The most immediate is to redesign the reward and action costs so that reward hacking is structurally impossible, then re-run the Crisis scenario as a clean test of whether the learning agent can be made as reliable as the FSM. Second, shrinking the state space towards the original 54-state design, or adopting a function approximator such as a deep Q-network that generalises across states [10], would address the coverage problem from the two opposite directions. Third, evaluation should move to fixed-length, multi-seed runs and, ideally, to genuinely adversarial settings with a second agent or a human opponent, which is the only way to exercise the military dimension the current runs leave dark. Fourth, a real human-alignment study, recording a knowledgeable player over the same scenarios and measuring agreement season by season, would turn the coarse correctness check into the evidence the educational framing ultimately needs.

5.7 Conclusion

This project set out to build and compare two autonomous agents for the JominiEngine, and it did. Both connect through the ordinary client interface with no server privilege, both pass a fifteen-case decision-correctness check, and both play complete multi-season games. The rule-based FSM proved to be a dependable steward: it keeps a growing fief network administered and solvent, holds its dynasty, and behaves the same way every time. The Q-learning agent showed that a policy learned from nothing but a scalar reward can match the FSM in ordinary conditions and can even recover a realm from debt, but also that the same adaptivity makes it brittle, discovering and exploiting a zero-cost loophole the moment the reward permitted it. The most valuable outcome is not a winner but an understanding: in a live historical economy, the hard problems are reward design and robustness, not the choice of learning algorithm, and a bounded evaluation score can

conceal exactly the behaviour that matters most. Those lessons, and the more robust engine left behind, are the foundation the next agent for this world can build on.

References

[1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete Problems in AI Safety. *arXiv preprint arXiv:1606.06565* (2016). https://arxiv.org/abs/1606.06565.

[2] David Bond, Hans-Wolfgang Loidl, and Sandy Louchart. 2015. Design and Implementation of the Jomini Engine: Towards a Historical Massively Multiplayer Online Role-Playing Game. In *Proceedings of the Games and Learning Alliance Conference (GALA 2015) (Lecture Notes in Computer Science)*. Springer, 441–451. doi:10.1007/978-3-319-40216-1_49

[3] Frontiers in Computer Science. 2021. The Evaluation of AI in Games Research. *Frontiers in Computer Science* (2021). doi:10.3389/fcomp.2021.774763

[4] James Paul Gee. 2003. *What Video Games Have to Teach Us About Learning and Literacy.* Palgrave Macmillan, New York.

[5] Marc Gravell. 2023. protobuf-net: Protocol Buffers for .NET. https://github.com/protobuf-net/protobuf-net. Accessed: 2026-07-14.

[6] Michael Lidgren Grønbæk. 2019. Lidgren.Network: A Networking Library for .NET. https://github.com/lidgren/lidgren-network-gen3. Accessed: 2026-07-14.

[7] Hans-Wolfgang Loidl and Sandy Louchart. 2016. The Jomini Engine: A Historical MMORPG Framework (draft). https://www.macs.hw.ac.uk/~hwloidl/publications/pwh16.pdf. Playing with History 2016. Accessed: 2026-07-14.

[8] David R. Michael and Sean Chen. 2005. *Serious Games: Games That Educate, Train, and Inform.* Course Technology / Thomson Learning, Boston, MA.

[9] Ian Millington and John Funge. 2009. *Artificial Intelligence for Games* (2nd ed.). Morgan Kaufmann, Burlington, MA.

[10] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. 2015. Human-level Control through Deep Reinforcement Learning. *Nature* 518 (2015), 529–533. doi:10.1038/nature14236

[11] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press, Cambridge, MA.

[12] Christopher J. C. H. Watkins. 1989. *Learning from Delayed Rewards.* Ph. D. Dissertation. King's College, University of Cambridge.

[13] Christopher J. C. H. Watkins and Peter Dayan. 1992. Q-Learning. *Machine Learning* 8 (1992), 279–292. doi:10.1007/BF00992698

A Project Management

The project ran across the second semester and summer period following the earlier research report. The implementation phase divided into three passes: bringing the agent up to a stable connection and a working FSM against the live server; adding the Q-learning agent and the evaluation harness; and a robustness pass, unplanned at the outset, that diagnosed and fixed the protocol and engine faults described in Section 3.3 once sustained multi-season runs began to fail. The evaluation runs were captured last, once the agents could complete long runs without crashing.

A.1 Deviations from the Plan

Two deviations from the design report are worth recording. First, the Q-learning state space grew from the planned 54 states to a twelve-feature discretisation (Section 3.6); the effect on learnability is discussed in Section 5.2. Second, robustness against the engine's real behaviour, barely mentioned in the plan, became a major work item and is reported as a first-class result in Section 5.4 rather than hidden as routine debugging.

B Generative AI

Generative AI tools were not used during the preparation of this dissertation. All intellectual content originates with the author. No AI-generated content was submitted, and no AI tool was used to produce or fabricate experimental results. This declaration is made in accordance with Heriot-Watt University's policy on the use of generative AI in assessed work.